Lab experiment-2

2. 8-Queens Problem

Aim

To place eight queens on an 8×8 chessboard so that no two queens attack each other.

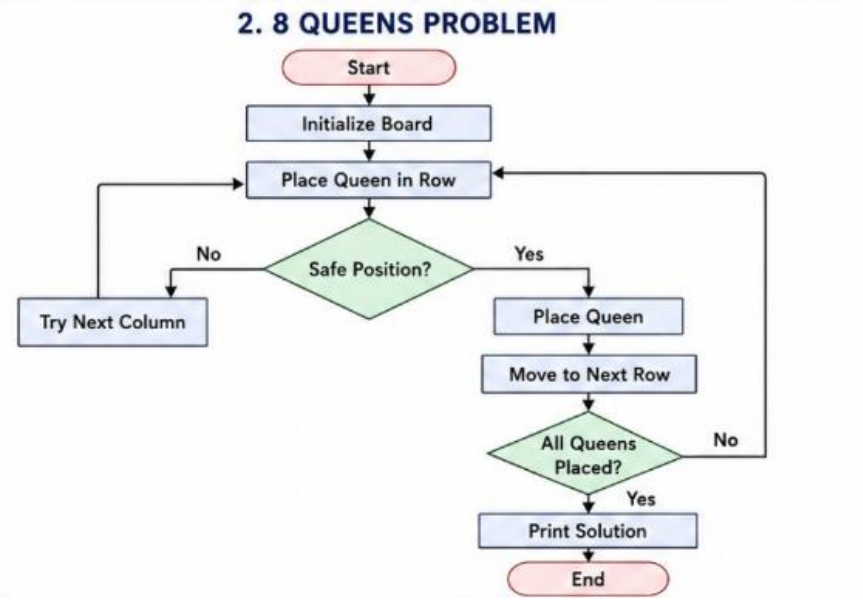
Objective

To find a valid arrangement of eight queens using the Backtracking algorithm.

Algorithm

1. Start from the first row.
2. Place a queen in a safe column.
3. Move to the next row.
4. If no safe position exists, backtrack.
5. Repeat until all queens are placed.
6. Print the solution.

Flowchart



Code

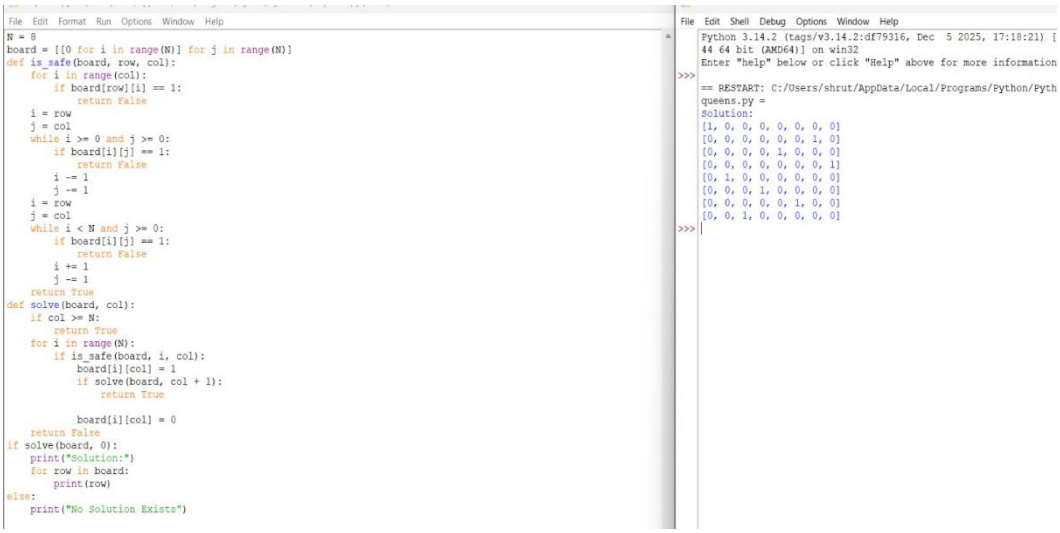
```
from collections import deque
goal = [1, 2, 3,
        4, 5, 6,
        7, 8, 0]
def get_neighbors(state):
    neighbors = []
    index = state.index(0)

    moves = {
        0: [1, 3],
        1: [0, 2, 4],
```

```
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
    }
    for move in moves[index]:
        new_state = state[:]
        new_state[index], new_state[move] = new_state[move], new_state[index]
        neighbors.append(new_state)
 return neighbors
def solve(start):
    queue = deque([(start, [])])
    visited = []
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.append(state)
        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))
    return None
```

```
start = [1, 2, 3,
         4, 5, 6,
         0, 7, 8]
solution = solve(start)
if solution:
    print("Solution Steps:")
    for step in solution:
        print(step[0:3])
        print(step[3:6])
        print(step[6:9])
        print()
else:
    print("No Solution Found")
```

Output



Result

The program successfully places all eight queens without conflicts.

Conclusion

Backtracking efficiently finds a valid solution for the 8-Queens problem.